QA CASE STUDY 02

API Testing & Validation

Cashless POS System

An anonymized QA case study demonstrating API, functional, negative, data validation, and regression testing using Postman.

Role: QA Analyst
Testing: API, Functional, Negative, Regression
Tool: Postman
Validation: API Response vs. UI

CONFIDENTIAL PROJECT | ANONYMIZED FOR PORTFOLIO

Confidentiality

This case study is based on professional QA experience and has been anonymized for portfolio purposes. Company names, application-specific information, screenshots, credentials, source code, proprietary workflows, internal data, and implementation-specific details have been excluded to protect confidential information.

Project Overview

The Cashless POS system manages product information through REST APIs that support core product management operations.

The API Layer Handles

- GET — retrieve product information
- POST — add new products
- PUT — update existing products
- DELETE — delete products

As QA Analyst, I used Postman to validate these endpoints and verify request and response behavior, including status codes, headers, authentication, request data, and returned product information.

I also compared API results with the POS application UI to confirm that backend operations were correctly reflected in the user-facing product workflow.

> *QA Focus: Validate that API operations perform the intended business action and maintain consistency between the API response and the POS application.*

QA Objective

Validate that product-related APIs correctly process requests, return expected responses, and maintain consistent data between the API and the application UI.

- API Functionality — verify GET, POST, PUT, and DELETE operations.
- Data Accuracy — validate request payloads and returned product data.
- Response Validation — verify status codes, response content, and expected API behavior.
- UI Consistency — confirm API changes are correctly reflected in the POS application.
- Regression — ensure API changes do not negatively affect existing product workflows.

My QA Contributions

- API Testing — executed GET, POST, PUT, and DELETE requests using Postman.
- Request Validation — validated request bodies, headers, authentication tokens, and input data.
- Response Validation — checked status codes, response body, and returned product data.
- Negative Testing — tested invalid requests and unexpected input conditions.
- UI/API Validation — compared API results against product behavior shown in the POS application.
- Defect Management — documented API defects, worked with developers, and retested fixes.

Confidentiality

This case study is based on professional QA experience and has been anonymized for portfolio purposes. Company names, application-specific information, screenshots, credentials, source code, proprietary workflows, internal data, and implementation-specific details have been excluded to protect confidential information.

- Regression Testing — validated related product workflows after API changes and fixes.

API Test Scope

The API supported the following core product operations: GET (retrieve products) → POST (add product) → PUT (update product) → DELETE (delete product).

Validation Flow

Request → API Processing → Response → Data Validation → UI Validation.

Test Strategy

A combination of test types was used to validate API behavior and its impact on the POS application.

- Functional — validated that each endpoint performed its intended product operation.
- Negative — tested invalid inputs and unsuccessful request scenarios to verify that the API handled errors appropriately.
- Data Validation — verified that product information returned by the API matched the expected product data.
- Response Validation — validated status codes, response content, and returned product information.
- API/UI Consistency — compared API results with the corresponding product behavior shown in the POS interface.
- Regression — validated related product workflows after API changes and fixes.

Sample API Test Coverage

ID	Method	Scenario	Expected Result
API-001	GET	Retrieve products	Product list returned
API-002	GET	Retrieve product data	Correct product details returned
API-003	POST	Add new product	Product created successfully
API-004	PUT	Update existing product	Existing product updated
API-005	DELETE	Delete product	Product removed successfully
API-006	POST	Invalid product data	Request rejected appropriately
API-007	PUT	Invalid update data	Request rejected appropriately
API-008	GET	Validate returned data	API data matches expected UI data
API-009	DELETE	Delete and verify UI	Product no longer appears

Confidentiality

This case study is based on professional QA experience and has been anonymized for portfolio purposes. Company names, application-specific information, screenshots, credentials, source code, proprietary workflows, internal data, and implementation-specific details have been excluded to protect confidential information.

These are representative portfolio scenarios based on the documented API testing scope.

Defects Identified

During API and UI validation, key issues were identified across product management workflows.

Defect 01 — Product Not Displayed Correctly

Severity: High Priority: High

Product data returned or created through the API was not reflected correctly in the POS application.

Expected: Product information should be correctly displayed in the POS UI.

Actual: Product was not showing as expected in the application.

Impact: Product information could become inconsistent between the API and the application UI.

QA Actions: Reproduce → Validate API → Verify UI → Report → Retest → Regression.

Defect 02 — PUT Request Created a New Product Instead of Updating

Severity: High Priority: High

When updating an existing product using a PUT request, the system created a new product instead of updating the existing record.

Expected: The existing product should be updated with the submitted values.

Actual: A new product was created instead of updating the existing product.

Impact:

- Duplicate product records
- Incorrect product information
- Data inconsistency
- Potential issues with downstream POS operations

QA Validation Flow: Existing Product → PUT Request → API Response → Product Data → POS UI.

Sample API Bug Report (BUG-API-001)

Field	Detail
Title	PUT request creates a new product instead of updating existing product
Severity / Priority	High / High
Environment	UAT
Initial Test	Failed
Retest	Passed

Confidentiality

This case study is based on professional QA experience and has been anonymized for portfolio purposes. Company names, application-specific information, screenshots, credentials, source code, proprietary workflows, internal data, and implementation-specific details have been excluded to protect confidential information.

Field	Detail
Status	Resolved

Reproduction steps: select an existing product, send a PUT request with updated product information, review the API response, retrieve the product list, then verify the product in the POS UI.

Expected Result: The existing product should be updated with the new information.
Actual Result: A new product was created instead of updating the existing product.

> *Result: FAILED (initial test) — resolved after fix and retest.*

Acceptance Criteria — Update Existing Product

Given an existing product is available, when a valid PUT request is submitted using the product identifier, then the existing product should be updated, no duplicate product should be created, and the updated product information should be reflected in the POS UI.

Defect Lifecycle

From detection to validation: Test → Defect Identified → Azure DevOps / Defect Log → Developer Investigation → Fix Implemented → Retest → Regression → Release Validation.

> *QA Principle: a defect is not considered complete after the initial fix. The affected scenario must be retested, followed by regression testing of related functionality.*

Sample Retest Task

Objective: verify that the product update defect was resolved and that the existing product is correctly updated without creating a duplicate record.

ID	Retest	Expected Result	Result
RT-API-001	Update existing product	Existing product updated	Pass
RT-API-002	Retrieve updated product	Updated data returned	Pass
RT-API-003	Verify product in UI	Updated product displayed	Pass
RT-API-004	Check duplicate record	No duplicate created	Pass

Confidentiality

This case study is based on professional QA experience and has been anonymized for portfolio purposes. Company names, application-specific information, screenshots, credentials, source code, proprietary workflows, internal data, and implementation-specific details have been excluded to protect confidential information.

Retest result: PASS — the PUT request correctly updated the existing product without creating a duplicate record, and the updated product information was reflected in the POS UI.

API Regression

Regression scope covering the full set of product management workflows.

ID	Regression Test	Expected Result	Result
REG-API-001	Retrieve products	Product list loads correctly	Pass
REG-API-002	Add product	New product is created	Pass
REG-API-003	Update product	Existing product is updated	Pass
REG-API-004	Delete product	Product is removed	Pass
REG-API-005	Product UI display	API changes reflected in UI	Pass
REG-API-006	Product data consistency	API and UI data remain consistent	Pass

Regression result: PASS — the API fixes did not introduce regressions across the tested product management workflows.

QA Traceability

Requirement	Test Case	Defect	Validation
Retrieve products	API-001, API-002	Defect 01	Regression
Add product	API-003	Defect 01	Regression
Update existing product	API-004	Defect 02	RT-API-001 to RT-API-004
Delete product	API-005	None	REG-API-004
API/UI consistency	API-008	Defect 01	REG-API-005, REG-API-006

Confidentiality

This case study is based on professional QA experience and has been anonymized for portfolio purposes. Company names, application-specific information, screenshots, credentials, source code, proprietary workflows, internal data, and implementation-specific details have been excluded to protect confidential information.

QA Validation Approach

- 1. Verify the request — confirm the correct method, headers, authentication, and request data are used.
- 2. Validate the response — check status code, response body, and returned product information.
- 3. Validate business behavior — confirm the API actually performs the intended product operation.
- 4. Verify UI consistency — confirm API changes are correctly reflected in the POS application.

Key QA Insight

A successful API response does not always mean the business operation succeeded.

QA must validate the actual business behavior and data consistency, not only the HTTP response. For example, a PUT request returning success does not automatically mean the existing product was updated correctly — the actual product state and the UI must also be validated.

Test Execution Summary

- 9 API test scenarios documented
- 4 retest scenarios documented
- 6 regression scenarios documented
- 2 key defects identified
- 100% of the documented retest scenarios passed
- 100% of the documented regression scenarios passed

These figures describe the documented portfolio coverage, not necessarily the complete original project test inventory.

QA Recommendation

Based on the completed retesting and regression validation, the tested product API workflows met the documented acceptance criteria. The updated product behavior was validated through API responses and corresponding POS UI behavior.

Confidentiality

This case study is based on professional QA experience and has been anonymized for portfolio purposes. Company names, application-specific information, screenshots, credentials, source code, proprietary workflows, internal data, and implementation-specific details have been excluded to protect confidential information.

Release Recommendation: PASS — based on the tested scope and documented results, the feature was considered ready to proceed to release validation.

Outcome & Key Takeaways

Testing identified inconsistencies between API behavior and the expected product behavior in the POS application. The identified defects were documented and coordinated with development. After fixes were implemented, the affected functionality was retested and related product workflows were regression tested.

Key Takeaways

- API validation is more than status codes — a successful HTTP response does not automatically mean the intended business operation was completed correctly.
- UI and API should be consistent — backend changes should produce the expected result in the user-facing application.
- Defects need retesting — a fix must be validated against the original issue before being considered resolved.
- Regression protects existing functionality — changes to one API operation can affect related product workflows.

Collaboration

Role	Contribution
QA Analyst	API testing, defect reporting, retesting, regression
Developer	Investigation and implementation of fixes
Project Team	Coordination and validation

Confidentiality

This case study is based on professional QA experience and has been anonymized for portfolio purposes. Company names, application-specific information, screenshots, credentials, source code, proprietary workflows, internal data, and implementation-specific details have been excluded to protect confidential information.

Skills and Tools

QA Skills

- API testing
- Functional testing
- Negative testing
- Data validation
- Response validation
- UI/API validation
- Defect management
- Retesting
- Regression testing
- Test case design

Tools

Postman: API testing and validation

Swagger: API documentation and endpoint reference, where applicable

Azure DevOps: Defect tracking and test management, where applicable

Confidentiality

This case study is based on professional QA experience and has been anonymized for portfolio purposes. Company names, application-specific information, screenshots, credentials, source code, proprietary workflows, internal data, and implementation-specific details have been excluded to protect confidential information.